Codebase Change Summary — Week 7

Coverage-Based Testing

SE 413/513 — Software Testing

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 6, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time rather than being rebuilt from scratch each week. This document summarizes exactly what was added or changed for Week 7 and, more importantly, explains *why* each change exists — which concept from the Week 7 reading, *Coverage-Based Testing: Measuring and Interpreting Coverage with `coverage.py` and `pytest-cov`*, it demonstrates. Read this alongside the reading and `demo-testing-suite/README.md`.

Key Idea

Nothing from Weeks 1–6 was removed. This week does two things: it adds a small, self-contained project that reproduces every coverage report in the reading, and it applies the reading's method to the *real* suite, closing the two coverage gaps that have been open since Week 4. The two intentional `xfail` results and one intentional `FAIL` are unchanged.

2 What Changed At a Glance

File	Status	Purpose
The main suite: closing real gaps		
`tests/orders/test_order_service.py`	Modified	A declined-payment stub: `place_order()`'s `"Payment failed"` line had never run.
`tests/notifications/test_password_reset_service.py`	Modified	Two `SmtpEmailClient` tests against a patched `smtplib.SMTP` — no mail server needed.
`app/notifications/password_reset.py`	Modified	One justified `# pragma: no cover` (an abstract method); docstrings updated.
The reading, reproduced (not in the default run)		
`demos/coverage_reports/checkout_app/checkout.py`	New	The reading's running example, verbatim: every line number matches.
`demos/coverage_reports/tests/`	New	The v1, v2, coupons-only, and no-assertion suites from the reading.
`demos/coverage_reports/coverage_settings.toml`	New	The reading's `[tool.coverage.*]` configuration.
`demos/coverage_reports/README.md`	New	Every command, in the reading's order, with its expected result.
Tooling and documentation		
`requirements.txt`	Modified	Pins `coverage>=7.5` (the HTML report's Functions view).
`.gitignore`	Modified	Ignores `coverage.xml` and `coverage.json`.
`docs/DEMO_GUIDE_week07_coverage_based_testing.md`	New	Section-by-section lecture companion for the reading.
`README.md` (suite and repository)	Modified	Updated layout, coverage commands, expected output, and the Weekly Updates index.

Tip

Reinstall the requirements before running anything this week:

```
cd demo-testing-suite
pip install -r requirements.txt
```

3 The Reading's Reports, Reproduced

Location: `demos/coverage_reports/`

This folder is a miniature project laid out exactly like the reading's (`checkout_app/checkout.py`, `tests/test_checkout_v1.py`, ...), so that report paths and line numbers match the reading character for character. **Run its commands from inside the folder:**

```
cd demos/coverage_reports
```

Run	Result	Reading section
v1, `coverage run` / `report`	`TOTAL 81%` (test files included)	The Basic Report
v1, `--source=checkout_app, -m`	74%; `8, 16-17, 24, 28`	Showing What Was Missed
v1, `--branch`	66%; `BrPart 3`	Turning On Branch Coverage
v1, settings file (`--rcfile`)	exit 2: `66 < fail-under=90`	Making Settings Permanent
v1, `pytest --cov-fail-under=80`	3 passed, exit 1	Enforcing a Minimum
coupons-only	41%; `16->18`	Reading the `Missing` Column
v2	100%, 9 passed	Closing the Gaps
no-assertion test	100%, 1 passed	Limitation 1
v2 vs. `amount > 50` (live edit)	100%, 9 passed	Limitation 3

Common Pitfall

Do not add a docstring to `checkout_app/checkout.py` or reformat it. Every `Missing` entry in the reading names a line of this file, so moving a line breaks the whole demo.

Why `coverage_settings.toml`, not `pyproject.toml`

The reading's configuration turns on branch measurement, restricts `source`, and sets `fail_under = 90`. If the folder had a `pyproject.toml` containing it, *every* run would use those settings, and you could never see the 81% → 74% → 66% progression. So the settings live under a name nothing reads by default, and the "permanent settings" step names the file explicitly:

```
coverage run --rcfile=coverage_settings.toml -m pytest tests/test_checkout_v1.py
coverage report --rcfile=coverage_settings.toml
```

```
Coverage failure: total of 66 is less than fail-under=90
```

`pytest --cov --cov-config=coverage_settings.toml` reads the same file. In your own project, call it `pyproject.toml` and drop the flag.

4 From Report to Tests, on the Real Suite

Locations: `tests/orders/test_order_service.py`, `tests/notifications/test_password_reset_service.py`, `app/notifications/password_reset.py`

Since Week 5, the whole-suite report has shown two gaps:

```
app/notifications/password_reset.py      19      7      0      0    63%   34, 47, 50-56
app/orders/order_service.py               8      1      2      1    80%   37
```

Following the reading's "From Report to Tests" method, each gap was read as a *behavior* that had never been checked, not as a line number to hit:

Gap	Behavior never checked	What was done
`order_service.py`, line 37	A declined payment is reported as `"Payment failed"`	New `DecliningPaymentStub`: a second canned answer next to Week 4's `PaymentStub`
`password_reset.py`, lines 47, 50–56	`SmtpEmailClient` sends to the configured host, and closes the connection *even when sending fails* (the `finally`)	Two tests that patch `smtplib.SMTP` (Week 5's mocking), so no mail server is involved
`password_reset.py`, line 34	None: it is the abstract `EmailClient.send`, which only raises `NotImplementedError`	Excluded with `# pragma: no cover` and a comment giving the reason

```
pytest --cov=app --cov-branch --cov-report=term-missing:skip-covered
```

```
TOTAL     113      0     34      0   100%

22 files skipped due to complete coverage.
```

Tip

Compare the two decisions in `password_reset.py`. The abstract method is excluded because it has no behavior, and that reason fits in one comment that would pass code review. `SmtpEmailClient` was *not* excluded, even though it looked hard to test: being hard to test is not a reason to exclude code (Best Practice 7). A test double made it easy after all.

5 What 100% Still Does Not Tell You

Every `app/` module is now at 100% branch coverage. The demo folder shows why that is not the same as "fully tested":

- `tests/test_checkout_noassert.py` reaches 100% with **no assertions** (Limitation 1).
- In the v2 suite, `amount >= 50` is never true; the only True outcome of line 22 comes from `is_member` (Limitation 2).
- Changing `>=` to `>` on line 22 leaves v2 passing, at 100% (Limitation 3). Make the edit, run, then restore with `git checkout -- checkout_app/checkout.py`.

Common Pitfall

The same is true of the main suite. Week 1's `calculate_late_fee` bug and Week 3's shipping boundary bug both sit inside files that report 100%. Those defects are found by the `xfail` tests, not by the coverage number.

6 Updated: Expected Test Results

From the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
```

```
pytest -v
```

you should now see **72 passed, 2 xfailed, 1 failed**: three more passing tests than Week 6's baseline, and the same two `xfail` results and one `FAIL` as before, unrelated to this week's additions. `demos/coverage_reports/` is not part of that run (`pytest.ini` sets `testpaths = tests`); run it from inside the folder.

7 How This Connects to the Reading

Key Idea

Weeks 5 and 6 defined coverage criteria. This week is about using the tools: measure, read the gaps rather than the percentage, and turn each gap into a behavior-level test with a real assertion. The demo folder shows that method on the reading's own example. The two closed gaps show it on code you have been working with since Week 4.

8 Suggested Next Steps

1. From `demos/coverage_reports/`, reproduce the 74% and 66% numbers, then open the HTML report (`coverage html`) and find the three yellow lines (the reading's Exercise 3).
2. Write the two tests Exercise 4 asks for, then make the `>=` → `>` edit and confirm that one of them fails. Restore the file afterwards.
3. Before reading the new tests in `tests/orders/` and `tests/notifications/`, restore the Week 6 versions (see the demo guide), look at the two gaps, and write your own tests for them. Then compare.
4. Run `pytest --cov=app --cov-branch --cov-report=html` from the project root and use the Functions view to confirm that every function in `app/` has been entered. Then name one function whose 100% you would *not* trust, and explain why.